

Vypracovanie zadania - TemperatureService.Api

Riešenie pozostáva z častí:

1. Temperature API

Hlavná REST API aplikácia poskytujúca údaje klientovi.

API:

- komunikuje s externou Weather službou,
- využíva Redis cache,
- obsahuje background refresh mechanizmus,
- implementuje Bearer token autentifikáciu.

API používa Bearer token autentifikáciu.

```
"ApiAuth": {  
  "Token": "temperature-api-secret-token"  
}
```

2. Weather Mock Service

Pre potreby testovania bola vytvorená samostatná mock služba simulujúca externý weather provider.

Táto služba:

- vracia testovacie údaje o počasí
- simuluje náhodné chyby
- simuluje timeouty
- umožňuje testovať správanie API pri nedostupnosti externého systému

Mock API služba je samostatne deploynutá a TemperatureService.Api s ňou komunikuje cez HTTPS

Je nasadená na: <https://weather.janohorak.com/swagger>

```
"ApiAuth": {  
  "Token": "weather-mock-secret-token"  
}
```

3. TemperatureService.LoadSimulator

jednoduchá konzolová aplikácia, slúži na simuláciu záťaže a generovanie HTTP požiadaviek na Temperature API.

Aplikácia paralelne volá API endpointy pre jednotlivé mestá a umožňuje:

- simulovať súbežné požiadavky na API,

- overiť load balancing medzi viacerými Kubernetes podmi,
- sledovať správanie Redis cache,
- demonštrovať fungovanie distributed lock mechanizmu pri

So štruktúrovaným logovaním a identifikáciou Kubernetes podov (POD_NAME) je možné v logoch sledovať:

- ktorý pod spracoval konkrétnu požiadavku,
- že plánovaný refresh cache vykonáva vždy iba jedna inštancia aplikácie,
- že ostatné pody refresh správne preskakujú z dôvodu aktívneho distributed locku.

Load simulator slúži ako jednoduchý diagnostický a demonstračný nástroj pre lokálny development a Kubernetes prostredie.

Fungovanie Temperature API

Komunikácia prebieha nasledovne:

Klient => Temperature API => Redis Cache => Weather Mock Service

Redis cache

Pri požiadavke klienta API najskôr kontroluje Redis cache.

Ak sa dáta nachádzajú v cache

- údaje sa vrátia okamžite,
- externá služba sa nevolá.

Ak sa dáta v cache nenachádzajú

- API zavolá Weather Mock Service,
- získa nové údaje,
- uloží ich do Redis cache,
- následne ich vráti klientovi.

Týmto spôsobom:

- sa znižuje počet externých volaní,
- zlepšuje výkon API,
- znižuje závislosť od externého systému.

Background refresh service

Projekt obsahuje background service implementovanú pomocou:

BackgroundService

Táto služba:

- beží pravidelne v nastavenom intervale,
- obnovuje údaje v Redis cache,
- zabezpečuje dostupnosť aktuálnych dát.

Výhodou je, že:

- klient nemusí čakať na načítanie dát z externého systému,
- API vracia údaje rýchlejšie
- systém je odolnejší voči krátkodobým výpadkom externého provideru.

Docker a k8s

Build Docker image

`docker build -t temperatureservice-api:latest -f src/TemperatureService.Api/Dockerfile .`

Deploy do Kubernetes

`kubectl apply -f k8s/deployment.yaml`

`kubectl apply -f k8s/service.yaml`

`kubectl apply -f k8s/ingress.yaml`

Overenie

Pody:

`kubectl get pods`

Services:

`kubectl get services`

Ingress:

`kubectl get ingress`

Zmena počtu API inštancií:

`kubectl scale deployment temperature-api --replicas=3`

Logy:

`kubectl logs deployment/temperature-api`

Reštart deploymentu:

`kubectl rollout restart deployment/temperature-api`

Port forwarding:

`kubectl port-forward service/temperature-api 8080:80`